

CS F407: Artificial Intelligence Lab - Week 8 Report

Bayesian Networks and Autoregressive Language Models

Author: Samar Talwar

Course: CS F407 (Artificial Intelligence), BITS Pilani Goa Campus

Date: Semester 1, AY 2026-27

1. Introduction & Theoretical Background

This laboratory formalises language modeling within the mathematical framework of **Bayesian Networks (Probabilistic Graphical Models)**.

1.1 Autoregressive Chain-Rule Factorisation

For a sequence of discrete random variables $X_1, X_2, \dots, X_T$ taking values in a finite vocabulary $\mathcal{V}$, the joint probability distribution factors exactly via the chain rule of probability: \$

$$P(X_1, X_2, \dots, X_T) = P(X_1) \prod_{t=2}^T P(X_t \mid X_1, \dots, X_{t-1})$$$

In an unconstrained Bayesian network, each variable X_t has $t - 1$ parent nodes, requiring an exponentially growing conditional probability table (CPT) with $|\mathcal{V}|^{t-1} \times (|\mathcal{V}| - 1)$ independent parameters.

1.2 First-Order Markov Assumption

The first-order Markov model simplifies the dependency structure by assuming conditional independence of X_t from all prior tokens given the immediate predecessor X_{t-1} : \$

$$P(X_t \mid X_1, \dots, X_{t-1}) \approx P(X_t \mid X_{t-1})$$$

Graph topology: A linear chain $X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow \dots \rightarrow X_T$.

Total possible contexts: $|\mathcal{V}|$. Total CPT parameters: $|\mathcal{V}| \times (|\mathcal{V}| - 1)$.

1.3 Second-Order Markov Assumption

The second-order Markov model conditions each token on the preceding two tokens: \$

$$P(X_t \mid X_1, \dots, X_{t-1}) \approx P(X_t \mid X_{t-2}, X_{t-1})$$$

Graph topology: Every node X_t has two parents: $X_{t-2} \rightarrow X_t$ and $X_{t-1} \rightarrow X_t$.

Total possible contexts: $|\mathcal{V}|^2$. Total CPT parameters: $|\mathcal{V}|^2 \times (|\mathcal{V}| - 1)$.

2. Dataset & Vocabulary

The canonical training corpus consists of 6 sentences: 1. the cat sat on the mat 2. the cat sat on the rug 3. the dog sat on the mat 4. the dog ran to the park 5. the cat ran to the park 6. the dog sat on the rug

Vocabulary ($\mathcal{V}$)

With special boundary tokens `START_TOKEN = "<START>"` and `END_TOKEN = "<END>"`, the vocabulary has $|\mathcal{V}| = 12$ tokens: [`'<END>'`, `'<START>'`, `'cat'`, `'dog'`, `'mat'`, `'on'`, `'park'`, `'ran'`, `'rug'`, `'sat'`, `'the'`, `'to'`]

3. Detailed Answers to Lab Questions (1–14)

Question 1: Chain-Rule Factorisation for Text Generation

Question: *Why is chain-rule factorisation useful for generating text?*

Answer:

The exact joint distribution $P(\mathbf{X}_1, \dots, \mathbf{X}_T)$ is intractable to parameterise or sample directly over long sequences because the space of possible sentences grows as $|\mathcal{V}|^T$.

Chain-rule factorisation decomposes the intractable full-sequence joint probability into a sequence of conditional step-by-step distributions: $P(\mathbf{X}_1, \dots, \mathbf{X}_T) = \prod_{t=1}^T P(\mathbf{X}_t \mid \mathbf{X}_{<t})$

This formulation enables **autoregressive token-by-token generation**: 1. Sample initial token $w_1 \sim P(\mathbf{X}_1 \mid \langle \text{START} \rangle)$. 2. At each subsequent step t , condition on already generated prefix $w_{<t}$ and sample $w_t \sim P(\mathbf{X}_t \mid w_{<t})$. 3. Terminate when the special sequence boundary token `<END>` is emitted.

Thus, generation is reduced to repeated sampling from low-dimensional discrete conditional distributions.

Question 2: First-Order Markov Network Independence Assumption

Question: *What independence assumption is made by the first-order Markov network?*

Answer:

The first-order Markov assumption asserts **conditional independence**: $\mathbf{X}_t \perp\!\!\!\perp (\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_{t-2}) \mid \mathbf{X}_{t-1}$
 Equivalently: $P(\mathbf{X}_t \mid \mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_{t-1}) = P(\mathbf{X}_t \mid \mathbf{X}_{t-1})$
 In Bayesian network terminology, the single parent node $\mathbf{X}_{t-1}$ forms the complete **Markov blanket** for $\mathbf{X}_t$ with respect to the past history $\mathbf{X}_{<t-1}$, separating $\mathbf{X}_t$ from all earlier ancestral tokens.

Question 3: Conditional Probability Tables (CPTs) & Zero-Probability Transitions

Question: *Construct conditional distributions for `the`, `cat`, `dog`, `sat`, `ran`. What are zero-probability transitions?*

Answer:

From `results/cpt_first_order.json`, the maximum-likelihood estimated distributions are:

Context (w_{t-1})	Total Count	Transition (w_t)	Observed Count	Probability $P(w_t \mid w_{t-1})$	Exact Fraction
<code>the</code>	12	<code>cat</code>	3	0.2500	1/4
		<code>dog</code>	3	0.2500	1/4
		<code>mat</code>	2	0.1667	1/6
		<code>park</code>	2	0.1667	1/6
		<code>rug</code>	2	0.1667	1/6
<code>cat</code>	3	<code>sat</code>	2	0.6667	2/3

Context ()	Total Count	Transition ()	Observed Count	Probability	Exact Fraction
		ran	1	0.3333	1/3
dog	3	sat	2	0.6667	2/3
		ran	1	0.3333	1/3
sat	4	on	4	1.0000	1/1
ran	2	to	2	1.0000	1/1
on	4	the	4	1.0000	1/1
to	2	the	2	1.0000	1/1
mat	2	<END>	2	1.0000	1/1
rug	2	<END>	2	1.0000	1/1
park	2	<END>	2	1.0000	1/1
<START>	6	the	6	1.0000	1/1

Zero-Probability Transitions

A zero-probability transition is any word pair (w_i, w_j) that does not appear consecutively anywhere in the training corpus ($C(w_i, w_j) = 0$). - Total possible transitions: $|\mathcal{V}|^2 = 12^2 = 144$. - Observed non-zero transitions: 17. - **Zero-probability transitions count: $144 - 17 = 127$** (88.19% sparsity). - Examples: $P(\text{dog} \mid \text{cat}) = 0.0$, $P(\text{sat} \mid \text{the}) = 0.0$, $P(\text{mat} \mid \text{ran}) = 0.0$.

Question 4: Transition Counts Storage

Question: Where in the program are the transition counts stored?

Answer:

In `week08_bayesian_networks/src/markov_model.py`: - In `FirstOrderMarkovModel`: stored in `self.counts`, which is a nested dictionary mapping each observed context string w_{t-1} to a `collections.Counter` of successor tokens w_t (`dict[str, Counter[str]]`). - In `SecondOrderMarkovModel`: stored in `self.counts`, mapping a 2-token tuple context (w_{t-2}, w_{t-1}) to a `collections.Counter` of successor tokens (`dict[tuple[str, str], Counter[str]]`). - Exported in unrounded format to `results/counts_first_order.json` and `results/counts_second_order.json`.

Question 5: Computation of $P(X_t \mid X_{t-1})$

Question: Where is $P(X_t \mid X_{t-1})$ computed?

Answer:

In `week08_bayesian_networks/src/markov_model.py` within `FirstOrderMarkovModel.train()` (lines 92–98):

```
for prev_tok, next_counts in self.counts.items():
    total = sum(next_counts.values())
    self.cpt[prev_tok] = {}
    for next_tok, cnt in sorted(next_counts.items()):
        self.cpt[prev_tok][next_tok] = cnt / total
```

This implements maximum likelihood estimation (MLE): $P(w_j | w_i) = \frac{C(w_i, w_j)}{\sum_{k \in \mathcal{V}} C(w_i, w_k)} = \frac{C(w_i, w_j)}{C(w_i)}$ The computed probabilities are stored in `self.cpt`.

Question 6: Next-Token Selection: Greedy vs Sampling

Question: How does the program choose the next word?

Answer:

In `markov_model.py:predict_next()`, two decoding strategies are implemented:

1. **Greedy Decoding** (`method="greedy"`): Selects the token with the highest conditional probability: $w_t^* = \arg \max_{w \in \mathcal{V}} P(w | \text{context})$ Tie-breaking policy: If multiple tokens have identical maximum probability (e.g., $P(\text{cat} | \text{the}) = 0.25$ and $P(\text{dog} | \text{the}) = 0.25$), ties are broken deterministically by alphabetical order (lexicographically smallest token is selected).
 2. **Stochastic Sampling** (`method="sampling"`): Draws the next token proportionally to its conditional probability using the **inverse cumulative distribution function (inverse-CDF)** algorithm with an explicit `random.Random(seed)` instance: - Sort candidate items deterministically. - Sample uniform random variate $r \sim U(0, 1)$. - Accumulate CDF: $F(k) = \sum_{j=1}^k P(w_j | \text{context})$. - Select smallest k such that $r \leq F(k)$.
-

Question 7: Handling Unseen Contexts

Question: What happens if the program encounters an unseen context?

Answer:

An unseen context is one where the conditioning history was never observed in training ($C(\text{context}) = 0$). - The model detects `if context not in self.observed_contexts:` and explicitly returns `None`. - In `generate()`, receiving `None` triggers a controlled, safe termination of generation without throwing an unhandled exception. - In `sentence_probability()`, encountering an unobserved transition sets the transition factor to $P = 0.0$, the joint sentence probability to 0.0 , and the log-probability to $-\infty$ (`float('-inf')`), correctly reflecting zero support in the estimated distribution. - The model never invents hallucinated probabilities or crashes.

Question 8: Normalisation Invariant Failure (Sum = 0.87)

Question: If a context total is 0.87, what does this indicate about the implementation?

Answer:

A valid conditional probability distribution must satisfy the **Axioms of Probability**: $\sum_{w \in \mathcal{V}} P(w | \text{context}) = 1.0$

If the sum evaluates to $0.87 \neq 1.0$, it indicates a severe defect in the normalization or counting logic, such as: 1. **Omitted transition counts**: A valid transition occurred in the training text but was dropped during parsing or filtering. 2. **Incorrect denominator**: The normalization divided counts by an arbitrary constant or an outdated sum instead of the true context count $\sum_k C(\text{context}, w_k)$. 3. **Floating-point truncation / leakage**: Probability mass was assigned to an unindexed category that was dropped from the CPT dictionary. 4. **Incorrect boundary token handling**: `<END>` transitions were excluded from the normalizer sum.

Empirical verification: In `results/normalisation_checks.json`, the maximum absolute deviation across all observed contexts in both models is 0.00×10^0 (exact to machine precision).

Question 9: Model Predictions vs Human Linguistic Expectations

Question: How do model predictions compare with human linguistic expectations?

Answer:

- **Strengths:** Because the training sentences are grammatically well-formed English, all observed transitions follow English syntax (Determiner → Noun → Verb → Preposition → Determiner → Noun). - **Limitations:** - *No Semantic Memory:* In first-order modeling, after generating "the cat ran to the", the model conditions only on "the". Because "the" was followed by "mat" and "rug" in sitting contexts, the first-order model can generate "the cat ran to the mat" or "the cat ran to the rug" with probability $1/6$ each—valid English syntax, but combinations not in the training corpus. - *Lack of Long-Range Topic Coherence:* The model has zero knowledge of entity permanence, world semantics, or intent beyond the local n-gram window.

Question 10: Greedy vs Sampling Generation Analysis

Question: Compare greedy and sampling generation (variation & diversity).

Answer:

From results/generation.json and results/comparison.json:

Greedy Generation

- **First-Order Model:** Enters an **infinite periodic cycle**.
- Transition trace: <START> → the → cat (tie-break over dog) → sat (prob 2/3) → on (prob 1.0) → the (prob 1.0) → cat → ...
- Because first-order greedy transitions are a deterministic map $f: \mathcal{V} \rightarrow \mathcal{V}$ on a finite state space, revisiting state 'the' guarantees an infinite loop ['the', 'cat', 'sat', 'on']. The cycle detector safely halts generation.
- **Second-Order Model:** Deterministically generates "the cat sat on the mat", terminating at <END>.

Stochastic Sampling ($N = 100$ runs)

- **First-Order Model:** High output diversity (**37 distinct sentences** generated out of 100 runs). Only 16% of generated sentences match training sentences verbatim. It generates novel grammatical sentences such as "the cat ran to the rug", "the dog sat on the park", etc.
 - **Second-Order Model:** Zero diversity (**6 distinct sentences** generated out of 100 runs). **100% of generated sentences are verbatim copies** of the 6 training sentences.
-

Question 11: First-Order vs Second-Order Model Differences

Question: Detailed comparison of first-order and second-order models.

Answer:

Dimension	First-Order Markov Model	Second-Order Markov Model
Bayesian Net Topology	Linear chain: $X_{t-1} \rightarrow X_t$	2-parent DAG: $X_{t-2} \rightarrow X_t \leftarrow X_{t-1}$
Context Representation	Single token w_{t-1}	Tuple of 2 tokens (w_{t-2}, w_{t-1})
Observed Contexts	11	15
Non-Zero Parameters	17	19
Average Branching Factor	1.545	1.267

Dimension	First-Order Markov Model	Second-Order Markov Model
Training Log-Likelihood	-22.8874	-10.7506 (higher fit)
Greedy Behavior	Cycles infinitely (the cat sat on the...)	Terminates with corpus sentence
Verbatim Copy Rate ($N = 100$)	16.0%	100.0%
Distinct Sentences ($N = 100$)	37	6

Question 12: Context Length, Prediction Quality, and Data Sparsity

Question: Why does increasing context improve prediction but increase data sparsity?

Answer:

Increasing context order k (from bigram $k = 1$ to trigram $k = 2$ to n -gram k):

- Prediction Accuracy & Specificity:** Conditioning on longer history $X_{t-k:t-1}$ provides more semantic context, resolving ambiguities (e.g. knowing "ran to" uniquely predicts "the" and "to the" uniquely predicts "park"). This increases training likelihood (log-likelihood improves from -22.89 to -10.75).
- The Curse of Dimensionality (CPT Parameter Explosion):** The context state space grows exponentially as $|\mathcal{V}|^k$.
 - $k = 1$: $|\mathcal{V}|^1 = 12$ possible contexts. Observed = 11 (91.7% observed).
 - $k = 2$: $|\mathcal{V}|^2 = 144$ possible contexts. Observed = 15 (**only 10.4% observed; 89.6% unobserved**).
 - $k = 3$: $|\mathcal{V}|^3 = 1,728$ possible contexts.
- Generalisation Failure:** With higher order, unobserved contexts multiply rapidly. Any novel 2-word prompt at test time will have $P(\cdot \mid \text{context}) = \text{undefined}$, causing complete model failure without smoothing/backoff.

Question 13: Probabilistic Specification vs Naive Prompting

Question: Why is formal probabilistic specification preferable to naive prompting for language model behavior?

Answer:

Formal probabilistic specifications guarantee mathematical invariants such as strict probability distribution normalization ($\sum_w P(w \mid c) = 1.0$ verified across all observed contexts in `results/normalisation_checks.json`, `first_order.max_absolute_error`) and exact joint likelihood factorizations (`results/chain_rule_probabilities.json`). Unlike naive prompt engineering where model outputs are subject to opaque heuristic drift and unconstrained sampling, discrete graphical formulations provide complete transparency, as every transition probability maps directly to verifiable empirical training counts (`results/counts_first_order.json`). Furthermore, generation failure modes—such as deterministic cyclic trapping during greedy decoding (observed in `results/comparison.json`, `first_order.greedy_generation.cycle_detected`)—can be analytically diagnosed and mitigated through structured sampling policies or context expansion rather than ad-hoc trial-and-error prompting.

Question 14: Value of Conceptualising Language Models as Bayesian Networks

Question: What does thinking of language models as Bayesian networks add?

Answer:

Conceptualizing language models as Bayesian networks reveals that modern autoregressive architectures and classical n -gram models share an identical mathematical foundation: factorizing joint sequence probabilities into directed conditional distributions via the chain rule ($P(X_1, \dots, X_T) = \prod_{t=1}^T P(X_t \mid X_{<t})$). Structuring sequence generation as a directed acyclic graph makes explicit the Markov independence assumptions that govern what history is retained versus discarded, directly explaining why expanding context from first-order ($k = 1$) to second-order ($k = 2$) improves training log-

likelihood from -22.8874 to -10.7506 (results/comparison.json, training_log_likelihood). Additionally, this perspective establishes that standard neural training objectives like token-level cross-entropy loss are mathematically equivalent to minimizing negative log-likelihood under the autoregressive Bayesian network factorization.

4. Chain-Rule Factorisation: Detailed Sentence Probability Table

From results/chain_rule_probabilities.json:

ID	Sentence	First-Order Joint P	First-Order $\log P$	Second-Order Joint P	Second-Order $\log P$
1	the cat sat on the mat	$1/36 \approx 0.02778$	-3.5835	$1/6 \approx 0.16667$	-1.7918
2	the cat sat on the rug	$1/36 \approx 0.02778$	-3.5835	$1/6 \approx 0.16667$	-1.7918
3	the dog sat on the mat	$1/36 \approx 0.02778$	-3.5835	$1/6 \approx 0.16667$	-1.7918
4	the dog ran to the park	$1/72 \approx 0.01389$	-4.2767	$1/6 \approx 0.16667$	-1.7918
5	the cat ran to the park	$1/72 \approx 0.01389$	-4.2767	$1/6 \approx 0.16667$	-1.7918
6	the dog sat on the rug	$1/36 \approx 0.02778$	-3.5835	$1/6 \approx 0.16667$	-1.7918
Total	Training Corpus Log-Likelihood	—	-22.8874	—	-10.7506

5. Verification & Test Suite Summary

The test suite in week08_bayesian_networks/tests/test_markov_models.py enforces 15 automated test cases:

- 1. **Independent Exact Fraction Oracle:** 100% agreement between model CPTs and standalone fractions.Fraction computations.
- 2. **Normalisation Axiom:** All observed CPTs sum to 1.0 (max deviation $< 10^{-12}$).
- 3. **Hand-Checked Probabilities:** 6 manual fractions verified.
- 4. **Stochastic Sampling Convergence:** With $N = 20,000$ draws, empirical frequencies match theoretical CPTs within ± 0.02 .
- 5. **Greedy Cycle Detection:** Successfully traps first-order periodic loops and synthetic looping graphs.
- 6. **N-gram Invariant:** Every generated transition is validated as a legitimate training n-gram.
- 7. **Three Monkeypatch Mutation Tests:**
 - Mutation 1 (Unnormalized CPT): Caught by normalization checker.
 - Mutation 2 (Ignored Context in 2nd-order): Caught by exact oracle.
 - Mutation 3 (Corrupted Greedy Tie-Break): Caught by greedy determinism assertion.